

UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA
Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Bloque 11 · Recuperación de acceso, versión de sesión y reparto de credenciales · API_TEATRO

ESPACIO ACADÉMICO:
Seguridad en el Desarrollo de Software

PRESENTADO A:
Sergio Arley Puerto Moreno

AUTORES:
Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. Qué pedía el bloque

El Bloque 11 prepara el backend para la interfaz web que viene después, pero todavía sin construirla. La guía pide tres cosas sobre la base que ya teníamos, sin borrar nada:

- **Poder revocar un JWT.** Un token firmado sigue siendo válido hasta que vence, aunque la persona cambie su contraseña. La guía lo resuelve con una columna `version_sesion` en `usuarios`.
- **Recuperar el acceso sin depender del correo.** Por dos vías: un token temporal que autoriza un administrador (`password_reset_tokens`) y ocho códigos de respaldo que la persona genera y guarda (`codigos_recuperacion`). En los dos casos la base guarda solo el hash.
- **Ordenar qué credencial pide cada ruta.** El navegador no debe cargar con una API Key privada: el login tiene que ser público y los recursos tienen que funcionar con el JWT.

Después hay que comprobarlo todo: las verificaciones 01 a 08 sobre la base, las 25 pruebas del Paso 8, las consultas del Paso 9 sobre lo que dejaron las pruebas y el preflight de CORS.

Cómo lo adaptamos al teatro

La guía trae tres scripts SQL con su propia numeración. En el teatro ya teníamos `01` a `06`, así que los nombramos siguiendo esa serie y dejamos escrita la correspondencia en `database/README.md`:

- `07_version_sesion.sql` corresponde a `11_3_version_sesion.sql`;
- `08_password_reset_tokens.sql` corresponde a `11_4_password_reset_tokens.sql`;
- `09_codigos_recuperacion.sql` corresponde a `11_4_2_codigos_recuperacion.sql`.

Donde la guía habla de nueve tablas, en el teatro son once: las nueve que ya teníamos más las dos nuevas. La ruta de "mis recursos" del Paso 8 es `/api/boletas/mias`, y la ruta del administrador para emitir un token es `POST /api/auth/recuperacion/{usuarioId}`.

Repositorio: https://github.com/JuanK2550/API-TEATRO (rama `main`, commits `2e2024a` a `53b5373`).

Todas las figuras salen de ejecuciones reales contra nuestra base de Supabase. En ninguna aparecen tokens completos, API Keys, contraseñas, hashes, el host ni la cadena de conexión, y el correo del administrador va enmascarado como `<ADMIN_EMAIL>`.

2. Las migraciones, sobre la base en marcha

La guía es clara en esto: conservar los datos, no volver a correr el esquema base y no usar `DROP`, `TRUNCATE` ni `RESTART IDENTITY`. Aplicamos solo `07`, `08` y `09`, en ese orden, y antes y después corrimos las consultas del Paso 2 para comparar.



Figura 1. Paso 2: el estado de la base antes y después de las migraciones. Los conteos de cada tabla son idénticos.

El `diff` entre los dos estados no encontró ninguna diferencia: 3 roles, 1 usuario, 11 eventos, 16 funciones, 48 tarifas, 29 descuentos, 5 asistentes y 7 boletas. Los tres scripts además se pueden aplicar dos veces sin romper nada, porque usan `IF NOT EXISTS` y borran cada política antes de crearla.

Un detalle de la consulta BD-00D: al principio salió vacía y parecía que faltaba la relación `usuarios.auth_user_id → auth.users.id`. No faltaba. `information_schema` solo muestra lo que el usuario que consulta puede ver, y el esquema `auth` es de Supabase. Al consultar `pg_catalog` la relación apareció intacta, con `ON DELETE RESTRICT`.

La diferencia que decide si las tablas sirven

Los scripts de la guía habilitan RLS y revocan permisos a los roles públicos, y ahí terminan. Les basta porque en ese escenario la API se conecta como dueña de las tablas y con `BYPASSRLS`: para ella RLS no existe.

En el teatro no es así. Desde el Bloque 7 la API entra con el rol `api_teatro`, que no es dueño de nada y no se salta RLS. Si hubiéramos copiado los scripts tal cual, las dos tablas nuevas no habrían servido: la API habría recibido *permission denied* al escribir y cero filas al leer. Por eso los nuestros añaden tres cosas, y además revocan también a `service_role`, como hicimos con las vistas en el Bloque 7:

La adaptación: permisos para api_teatro — Bloque 11

Archivo: database/08_password_reset_tokens.sql

```
60  ALTER TABLE public.password_reset_tokens ENABLE ROW LEVEL SECURITY;
61
62  REVOKE ALL ON TABLE public.password_reset_tokens
63      FROM PUBLIC, anon, authenticated, service_role;
64
65  REVOKE ALL ON SEQUENCE public.password_reset_tokens_id_seq
66      FROM PUBLIC, anon, authenticated, service_role;
67
68  GRANT SELECT, INSERT, UPDATE, DELETE
69      ON public.password_reset_tokens TO api_teatro;
70
71  GRANT USAGE, SELECT
72      ON SEQUENCE public.password_reset_tokens_id_seq TO api_teatro;
73
74  DROP POLICY IF EXISTS api_teatro_total ON public.password_reset_tokens;
75  CREATE POLICY api_teatro_total ON public.password_reset_tokens
76      FOR ALL TO api_teatro USING (TRUE) WITH CHECK (TRUE);
```

Sin el GRANT sobre la tabla, el GRANT sobre la secuencia y la política TO api_teatro, la API recibiría permission denied. Los roles públicos y service_role quedan sin ningún privilegio.

API_TEATRO · Bloque 11 · Recuperación de acceso y reparto de credenciales

Figura 2. Fragmento de 08_password_reset_tokens.sql: el GRANT de tabla y secuencia y la política limitada a api_teatro. La 09 hace lo mismo.

La política es `USING (true)`, pero solo para `api_teatro`. La guía advierte contra las políticas permisivas para todos, y esta no lo es: ningún otro rol puede usarla. Lo comprobamos en la figura siguiente.

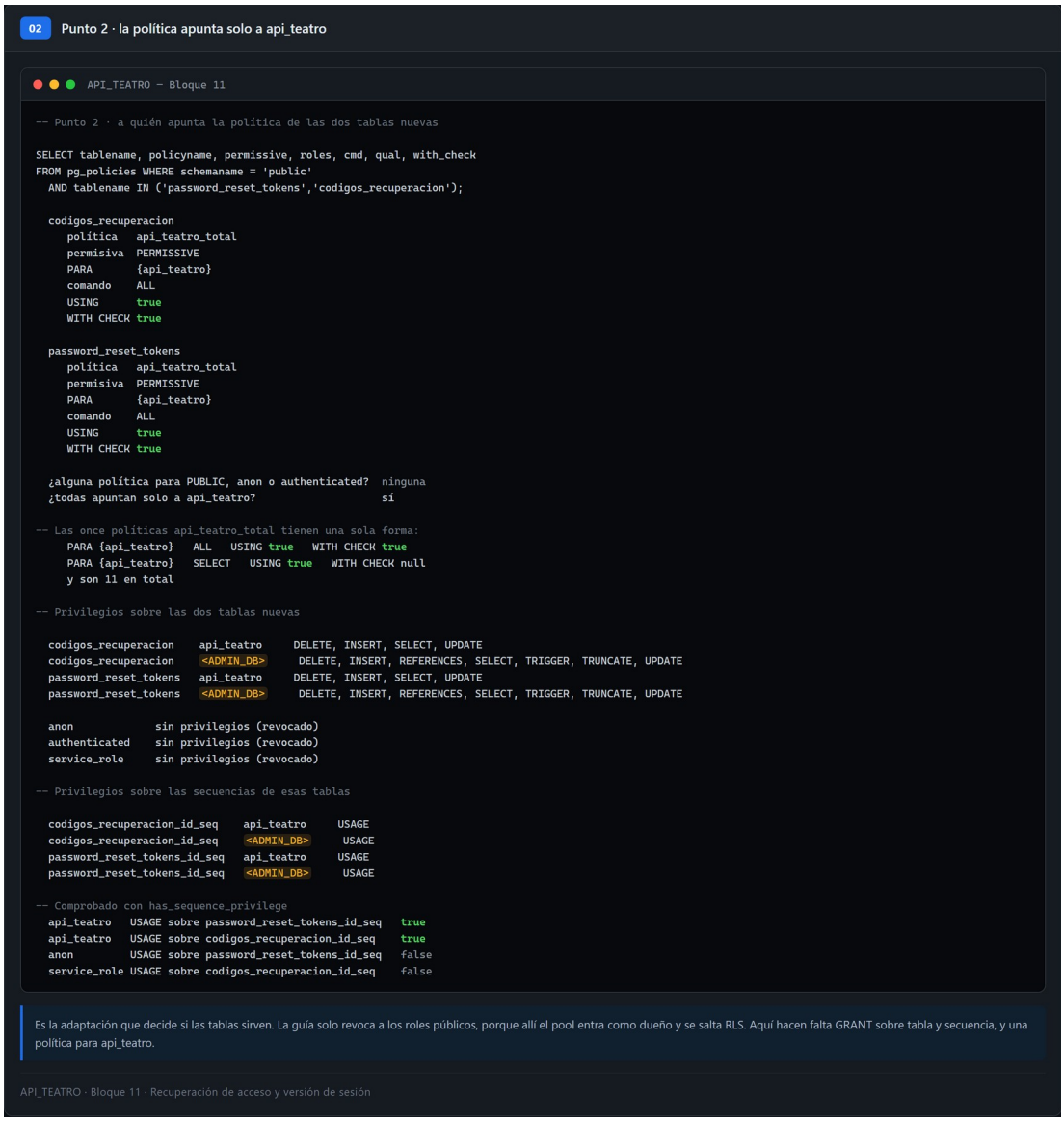


Figura 3. Políticas y privilegios de las dos tablas nuevas: todo para api_teatro y nada para anon, authenticated ni service_role.

3. Verificaciones 01 a 08

La verificación 01 confirma las once tablas con RLS habilitado, y la 02 muestra la columna nueva como `integer NOT NULL` con cero por defecto. Le agregamos un `CHECK (version_sesion >= 0)` que la guía no trae: una versión de sesión solo puede avanzar, nunca ser negativa.

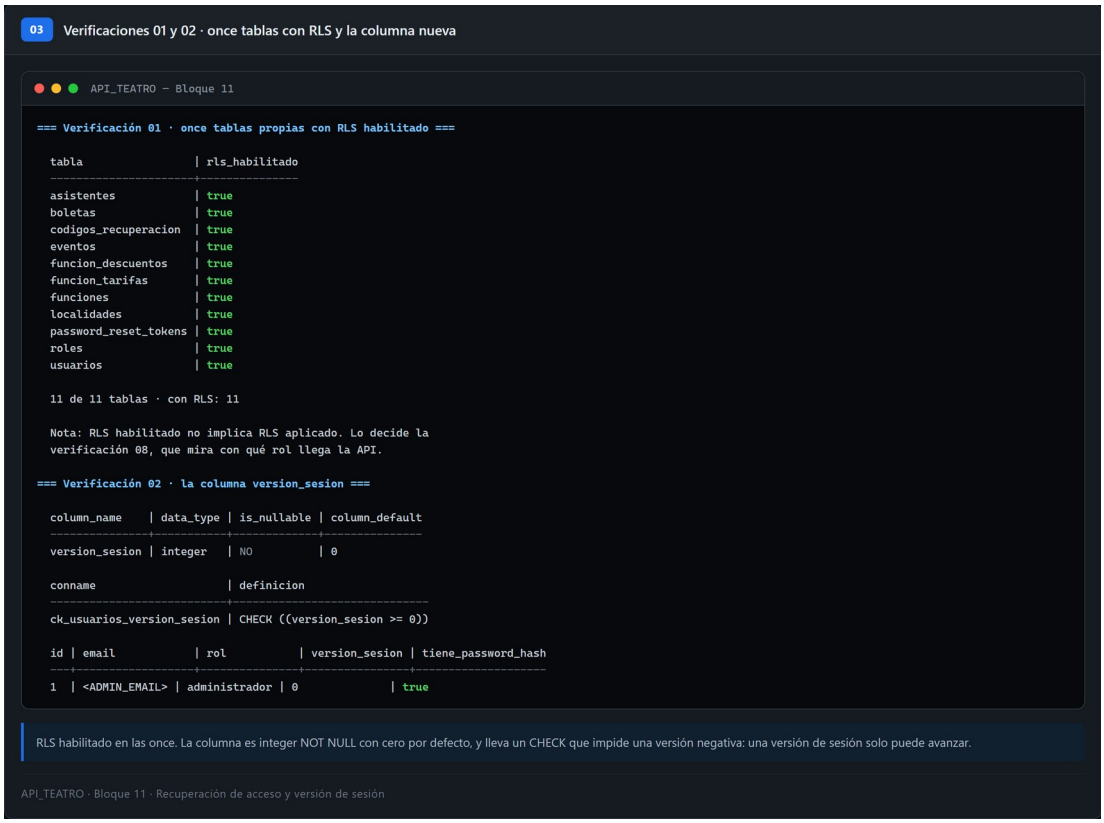


Figura 4. Verificaciones 01 y 02: once tablas con RLS y la columna version_sesion con su restricción.

Las verificaciones 03, 04 y 05 revisan columnas, claves foráneas, índices y restricciones de las tablas nuevas. `password_reset_tokens` apunta dos veces a `usuarios`: una por la cuenta que recupera el acceso y otra por el administrador que lo autoriza. La segunda no tiene `ON DELETE CASCADE` a propósito, para que borrar a un administrador no borre el rastro de lo que autorizó.



Figura 5. Verificaciones 03 a 05: columnas, claves foráneas (incluida la relación con auth.users), índices y restricciones.

La verificación 08: con qué rol llega de verdad la API

Es la advertencia más importante de la guía: **RLS habilitado no significa RLS aplicado**. Si la conexión de la API tiene `BYPASSRLS`, las políticas no se evalúan nunca. Por eso la consulta no se corre en el editor SQL de Supabase, sino desde el mismo `pg.Pool` que usa la API.

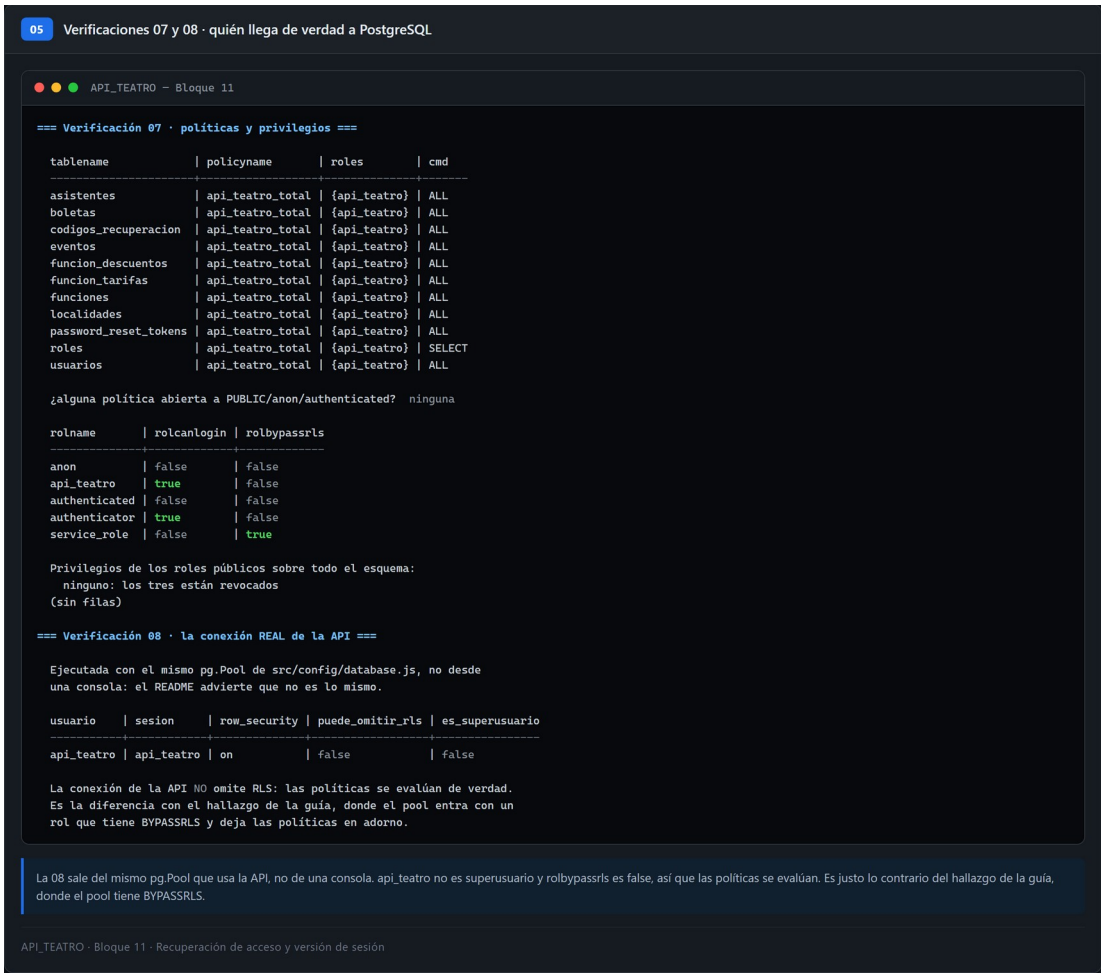


Figura 6. Verificaciones 07 y 08: once políticas, todas para api_teatro, y la conexión real de la API sin BYPASSRLS ni superusuario.

El resultado es `api_teatro`, con `rolbypassrls = false` y `rolsuper = false`. En la guía este punto queda como hallazgo pendiente, porque allí la conexión sí se salta RLS. En el teatro ese problema lo resolvimos en el Bloque 7, y aquí se ve que sigue resuelto.

4. Revocar un JWT con la versión de sesión

El problema es sencillo de explicar: un JWT no se puede borrar. Si alguien roba un token y la víctima cambia su contraseña, el token robado sigue sirviendo hasta que vence. La solución de la guía tiene dos partes:

- al firmar el token se guarda dentro la versión de sesión de la cuenta, en el campo `vs`;
- en cada petición el middleware lee la versión que hay en la base y la compara con la del token. Cada cambio o restablecimiento de contraseña suma uno, así que los tokens anteriores dejan de coincidir.



Figura 7. Fragmento de auth.middleware.js: el rol, el estado y la versión se leen de la base en cada petición.

Esto tiene un costo: el middleware hace una consulta a PostgreSQL en cada petición autenticada. Lo aceptamos porque sin consultar la base no hay forma de saber que la contraseña cambió. Y trae una ventaja extra: como el rol también sale de la base, desactivar una cuenta o bajarle el rol surte efecto de inmediato, sin esperar a que el token venza.

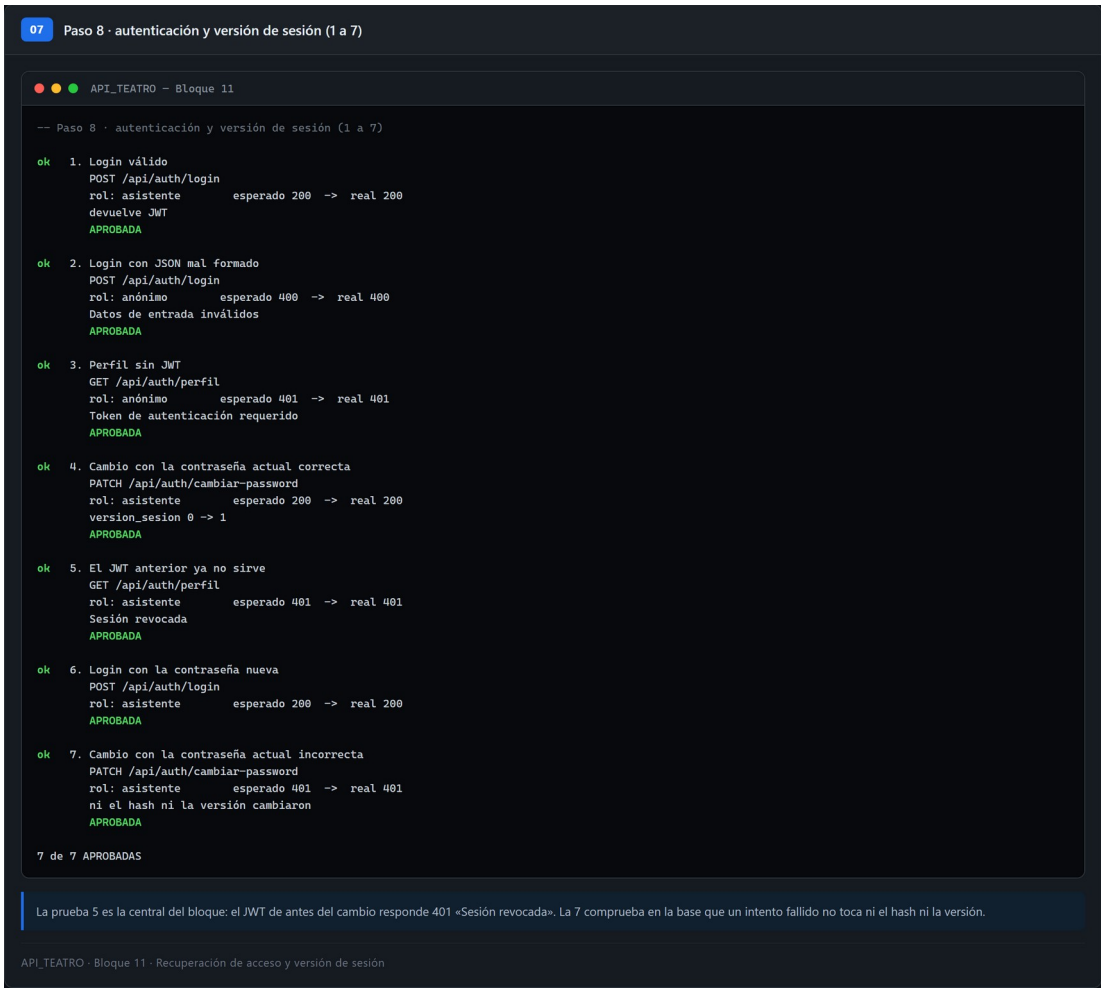


Figura 8. Paso 8, pruebas 1 a 7: login, cambio de contraseña y el JWT anterior rechazado con 401 Sesión revocada.

La prueba 5 es la que importa: después del cambio de contraseña (prueba 4, versión de 0 a 1) el token anterior recibe 401. La prueba 7 muestra el otro lado: con la contraseña actual equivocada el cambio se rechaza y ni el hash ni la versión se tocan.

5. Recuperar el acceso: token del administrador y códigos de respaldo

El token que autoriza un administrador

Cuando alguien pierde el acceso, un administrador verifica su identidad por fuera del sistema y emite un token aleatorio de 32 bytes que vence a los 15 minutos. La API lo muestra una sola vez y en la base guarda su SHA-256. Si alguien llegara a leer la tabla, solo encontraría hashes que no sirven para restablecer nada.

Restablecer con un token de un solo uso Bloque 11

Archivo: src/services/recuperacion.service.js

```
  return enTransaccion(async (consultar) => {
    const { rows } = await consultar(
      `SELECT t.id,
              t.usuario_id,
              t.fecha_expiracion,
              t.fecha_uso,
              u.activo,
              u.password_hash
         FROM public.password_reset_tokens t
         JOIN public.usuarios u ON u.id = t.usuario_id
         WHERE t.token_hash = $1
         FOR UPDATE OF t, u`,
      [huella(token)]
...
    // Un token que no existe, ya usado, vencido o de una cuenta desactivada se
    // responde igual: no se le dice al cliente cuál de las cuatro cosas pasó.
    const vencido =
      registro && new Date(registro.fecha_expiracion).getTime() <= Date.now();

    if (!registro || registro.fecha_uso || !registro.activo || vencido) {
      return { estado: "TOKEN_INVALIDO" };
    }

    if (await verificarPassword(passwordNueva, registro.password_hash)) {
      return { estado: "MISMA_CLAVE" };
    }

    await consultar(
      `UPDATE public.usuarios
         SET password_hash = $1, version_sesion = version_sesion + 1
         WHERE id = $2`,
      [passwordHash, registro.usuario_id]
    );

    await consultar(
      `UPDATE public.password_reset_tokens
         SET fecha_uso = NOW()
         WHERE usuario_id = $1 AND fecha_uso IS NULL`,
      [registro.usuario_id]
    );
```

Se busca por el hash, se bloquea la fila y se responde igual ante un token inexistente, usado, vencido o de una cuenta inactiva. Al restablecer sube la versión y el token queda marcado con fecha_uso.

API_TEATRO · Bloque 11 · Recuperación de acceso y reparto de credenciales

Figura 9. Fragmento de recuperacion.service.js: el token se busca por su hash, se bloquea la fila y se consume una sola vez.

Tres decisiones del servicio nos parecieron importantes:

- **Una sola respuesta para cuatro fallos.** Un token que no existe, ya usado, vencido o de una cuenta desactivada recibe el mismo 400. Así no se le da pistas a quien está probando tokens.
- **Bloqueo de filas.** El `FOR UPDATE` evita que dos peticiones simultáneas usen el mismo token.
- **Restablecer cierra todas las puertas.** Además de subir la versión, marca como usados los demás tokens vigentes y los códigos de respaldo de esa cuenta: quien perdió el acceso no debe conservar otra vía de entrada.

Los ocho códigos de respaldo

La persona, con su sesión abierta y confirmando su contraseña, genera ocho códigos que se le muestran una sola vez. Cada uno sirve para restablecer la contraseña una vez. Regenerarlos invalida el lote anterior, y la consulta de estado solo dice cuántos quedan, nunca cuáles.

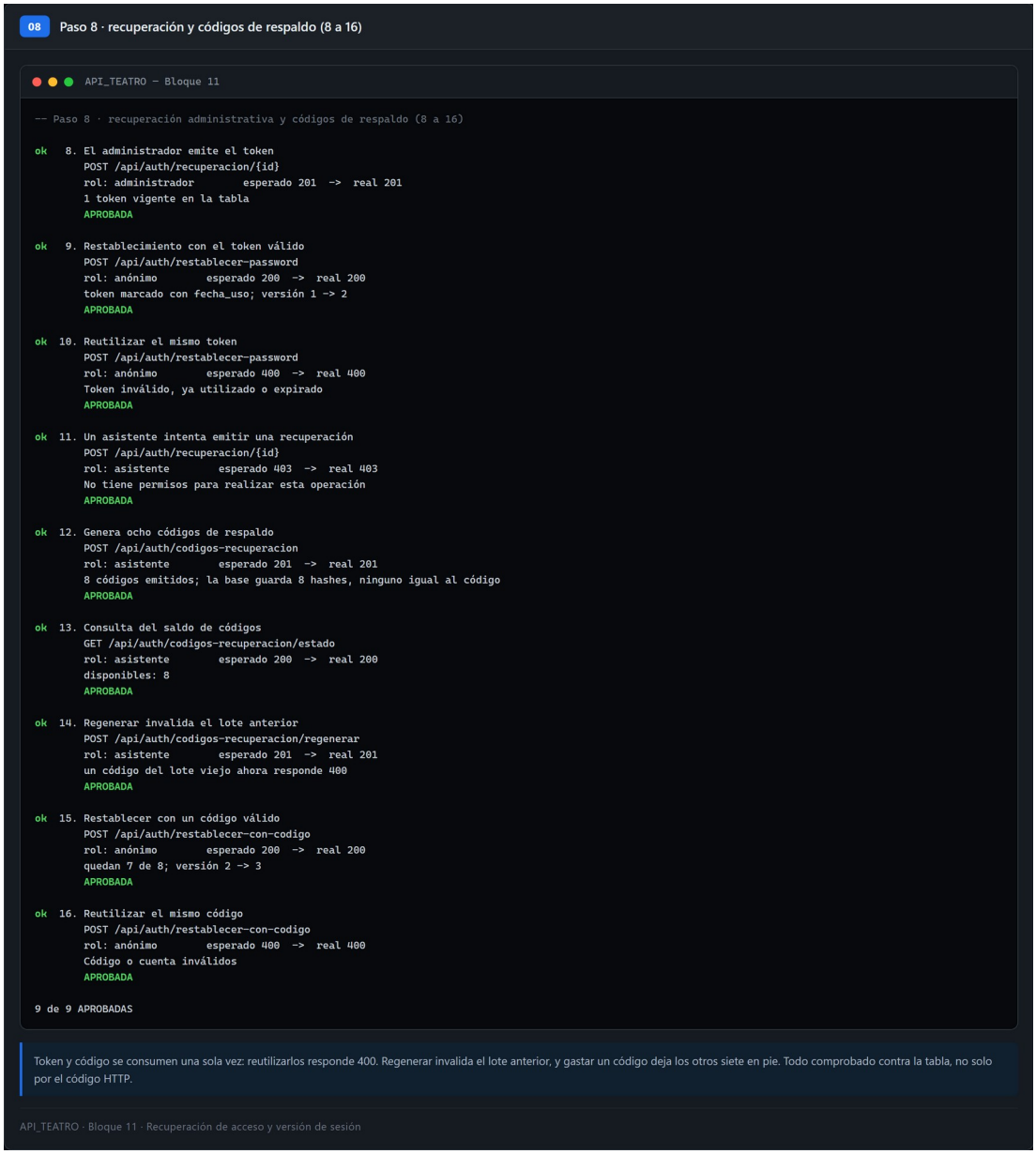


Figura 10. Paso 8, pruebas 8 a 16: el token y el código funcionan una vez y la segunda vez responden 400.

Las pruebas 10 y 16 son las que pide la guía: reutilizar el token o el código recibe un rechazo. La 11 comprueba RBAC: un asistente que intenta emitir una recuperación recibe 403.

Paso 9: lo que quedó en la base

Después de las pruebas volvimos a la base con las consultas del Paso 9, sin seleccionar nunca `token_hash` ni `codigo_hash`. Para que la captura fuera limpia, el Paso 9 corre con una cuenta propia justo después de la batería: se le emiten dos tokens, se restablece con uno y luego con un código.

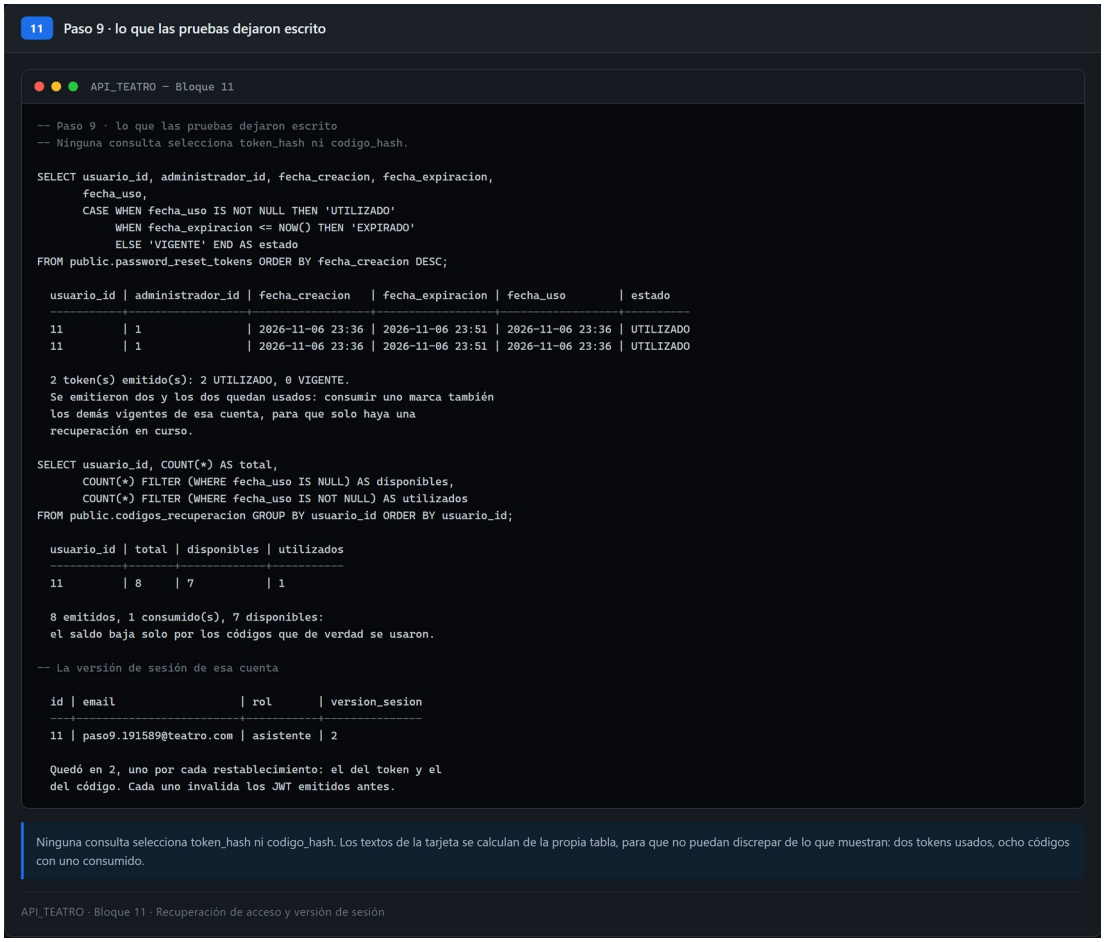


Figura 11. Paso 9: dos tokens, los dos UTILIZADO; ocho códigos con uno consumido; versión de sesión en 2.

Los dos tokens aparecen como `UTILIZADO` aunque solo se usó uno, porque consumir un token marca también los demás vigentes de esa cuenta. De los códigos queda 1 usado y 7 disponibles, y la versión quedó en 2: una por el restablecimiento con token y otra por el del código. Esa cuenta no hizo cambio de contraseña, por eso no llega a 3 como la cuenta de la prueba 15.

6. El reparto de credenciales: el navegador deja de recibir la API Key

Esta fue la parte del bloque que más nos hizo pensar. Hasta ahora el teatro exigía API Key en todo `/api`, incluido el login. Como la Sala (nuestra página web) también la necesitaba,

el servidor se la entregaba al navegador desde `/sala/config.js`. Y una clave que se le entrega a cualquier navegador no es secreta: se lee con el inspector.

En la primera pasada dejamos las pruebas 22 y 24 con 401, que era lo que daba el teatro, en vez del 400 y el 200 que pide la guía. Al revisarlo vimos que eso era cambiar el resultado esperado para que la prueba pasara, y que el problema de fondo era real. Así que lo corregimos en el código.

Tres capas

- **Público, sin API Key ni JWT:** `POST /auth/login`, `POST /auth/registro` y los nueve `GET` del catálogo (eventos, funciones, tarifas, ocupación y localidades).
- **Solo JWT:** perfil, cambio de contraseña, emisión de tokens, códigos de respaldo y todo lo de asistentes, boletas, usuarios y las escrituras del catálogo.
- **Solo API Key:** `/api/seguridad/cliente` y las dos rutas de restablecimiento, que no tienen sesión porque quien las usa perdió el acceso.

El portero: quién exige API Key y quién no — Bloque 11

Archivo: src/app.js

```
// Recursos cuyo router ya decide qué es público y qué exige JWT.
const RECURSOS = new Set([
  "/asistentes",
  "/boletas",
  "/eventos",
  "/funciones",
  "/localidades",
  "/usuarios"
]);

// Rutas de autenticación que se resuelven con JWT, sin API Key.
const AUTH_CON_SESION = new Set([
  "/auth/perfil",
  "/auth/cambiar-password",
  "/auth/recuperacion",
  "/auth/codigos-recuperacion"
]);

// Las dos públicas de verdad: quien entra o se registra todavía no tiene nada.
const AUTH_PUBLICAS = new Set(["/auth/login", "/auth/registro"]);

app.use("/api", (req, res, next) => {
  if (req.method === "POST" && AUTH_PUBLICAS.has(req.path)) return next();

  const recurso = "/" + req.path.split("/")[1];
  if (RECURSOS.has(recurso)) return next();

  const rutaAuth = "/" + req.path.split("/").slice(1, 3).join("/");
  if (AUTH_CON_SESION.has(rutaAuth)) return next();

  // Lo que queda: /seguridad/cliente y las dos rutas de restablecimiento, que
  // no tienen sesión porque quien las usa perdió el acceso.
  return validarApiKey(req, res, next);
});
```

Login y registro pasan sin clave; los recursos y las rutas con sesión los decide su router con JWT. Solo el cliente privado y las dos rutas de restablecimiento siguen pidiendo API Key.

API_TEATRO · Bloque 11 · Recuperación de acceso y reparto de credenciales

Figura 12. Fragmento de app.js: el portero que decide qué rutas piden API Key.

Dos decisiones que se apartan de la guía

El catálogo es público de solo lectura. En la guía, los recursos pasan sin API Key porque todos exigen JWT. En el teatro, esos mismos recursos incluyen la cartelera, que la Sala muestra a quien todavía no ha iniciado sesión. Pedir registro para ver qué hay en un teatro no tiene sentido, así que la dejamos pública con tres condiciones, todas comprobadas:

- la ocupación solo devuelve `localidadId`, `fila` y `numero`: qué butacas están tomadas, nunca de quién ni con qué boleta;
- lo inactivo no se muestra: un evento desactivado no sale en la lista y responde 404 por id, y solo aparecen las funciones `en_venta`, `agotada` o `en_curso`;
- no se exponen campos internos ni identificadores de usuario, y todas las escrituras siguen exigiendo JWT de administrador.

El registro también es público. La guía lo deja con API Key, pero la Sala crea cuentas desde el navegador: sin la clave, nadie podría registrarse. Lo tratamos como el login. El rol lo sigue poniendo el servidor (la prueba 33 pide `administrador` y la cuenta queda como `asistente`), y login y registro estrenan un límite de 10 intentos por ventana. Antes no tenían ninguno porque la API Key frenaba los intentos en serie; ahora el límite es la barrera.

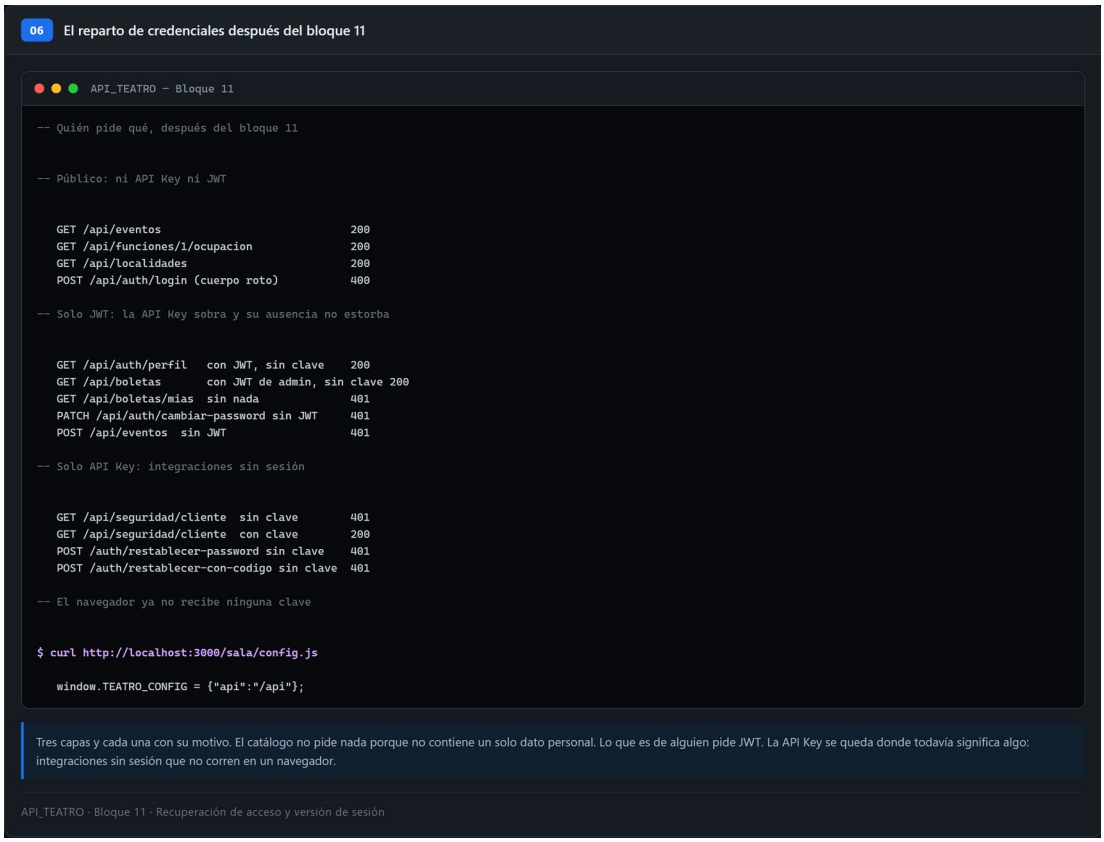


Figura 13. El reparto comprobado contra el servidor, capa por capa, y /sala/config.js ya sin la clave.

`/sala/config.js` ahora devuelve solo `{"api":"/api"}`. Como esa clave estuvo expuesta varios bloques, la dimos por pública y desactivamos el cliente "Aplicación Web": con esa clave las rutas que todavía piden API Key responden 403 «API Key deshabilitada».

Las pruebas 17 a 35

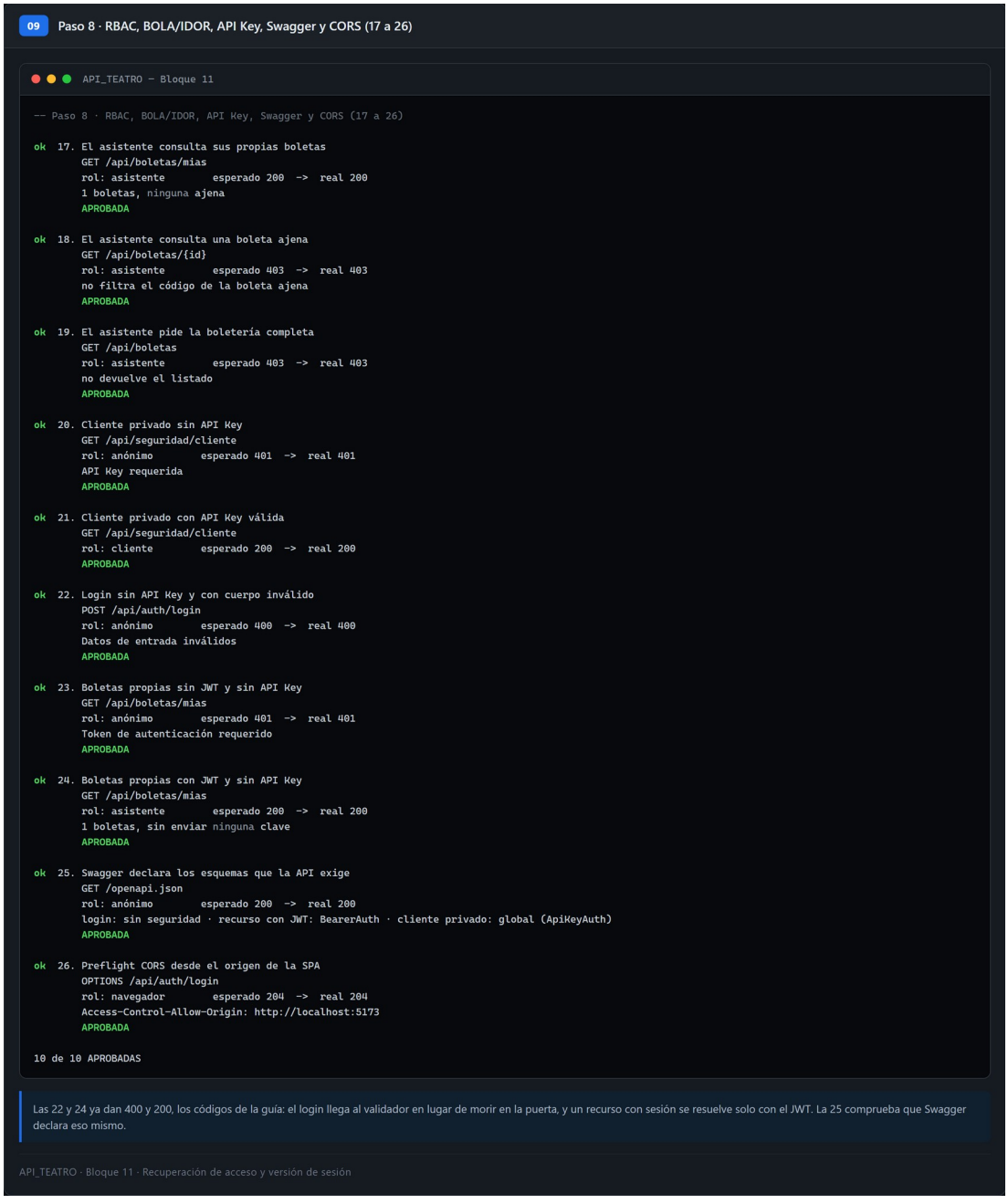


Figura 14. Paso 8, pruebas 17 a 26: BOLA/IDOR, cliente privado, login sin API Key, Swagger y preflight CORS.

Las pruebas 17 a 19 son las de autorización: un asistente ve sus boletas, recibe 403 ante una boleta ajena y 403 ante el listado completo. La 22 y la 24 ya dan los códigos de la guía, 400 y 200. La 25 revisa que Swagger declare lo que la API exige: el login sin seguridad,

los recursos con `BearerAuth` y el cliente privado con `ApiKeyAuth`. La 26 es el preflight CORS, que responde 204 con `Access-Control-Allow-Origin: http://localhost:5173`.

Agregamos nueve pruebas propias, de la 27 a la 35, para demostrar las dos decisiones de esta sección:

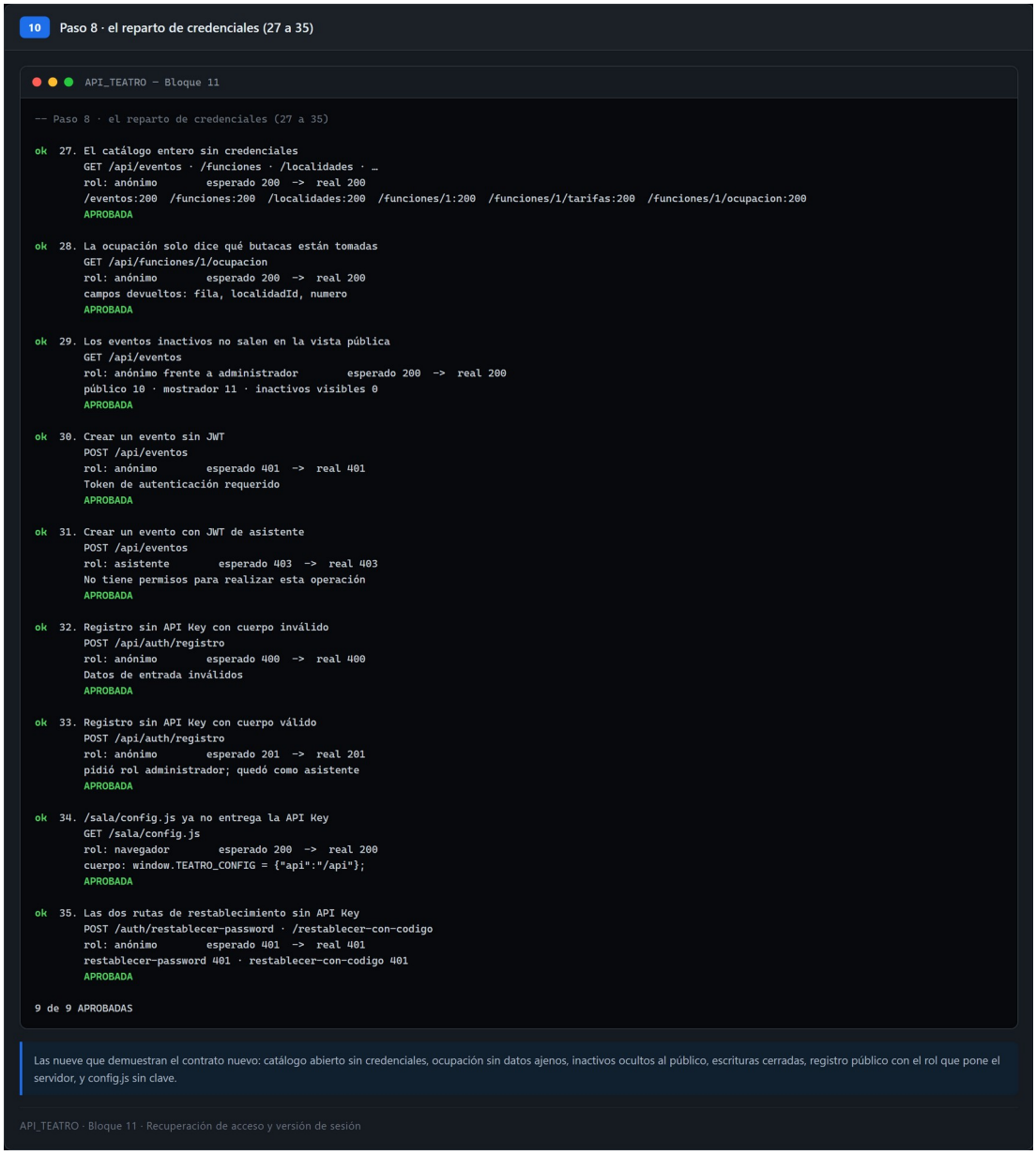


Figura 15. Pruebas 27 a 35: catálogo abierto, ocupación sin datos ajenos, inactivos ocultos, escrituras cerradas, registro público y config.js sin clave.

7. Matriz de pruebas

Esta es la tabla que pide el Paso 11. Las 35 quedaron APROBADAS. Del 1 al 26 son las de la guía (la 26 es el preflight CORS del Paso 10) y del 27 al 35 las que agregamos.

#	Prueba	Endpoint	Rol	HTTP	Observación
1	Login válido	POST /auth/login	asistente	200	Devuelve JWT
2	Login con JSON inválido	POST /auth/login	anónimo	400	Datos de entrada inválidos
3	Perfil sin JWT	GET /auth/perfil	anónimo	401	Token requerido
4	Cambio con contraseña correcta	PATCH /auth/cambiar-password	asistente	200	Versión 0 → 1
5	Usar el JWT anterior	GET /auth/perfil	asistente	401	Sesión revocada
6	Login con la contraseña nueva	POST /auth/login	asistente	200	—
7	Cambio con contraseña incorrecta	PATCH /auth/cambiar-password	asistente	401	Hash y versión intactos
8	Administrador emite el token	POST /auth/recuperacion/{id}	admin	201	1 token vigente
9	Restablecer con token válido	POST /auth/restablecer-password	anónimo	200	Token con fecha_uso; versión 1 → 2
10	Reutilizar el token	POST /auth/restablecer-password	anónimo	400	Token inválido o usado
11	Asistente intenta emitir un token	POST /auth/recuperacion/{id}	asistente	403	RBAC
12	Generar ocho códigos	POST /auth/codigos-recuperacion	asistente	201	8 hashes en la base
13	Saldo de códigos	GET /auth/codigos-recuperacion/estado	asistente	200	Disponibles: 8
14	Regenerar códigos	POST /auth/codigos-recuperacion/regenerar	asistente	201	El lote viejo da 400
15	Restablecer con un código	POST /auth/restablecer-con-codigo	anónimo	200	Quedan 7; versión 2 → 3
16	Reutilizar el código	POST /auth/restablecer-con-codigo	anónimo	400	Código inválido
17	Mis boletas con JWT	GET /boletas/mias	asistente	200	Ninguna ajena
18	Boleta ajena	GET /boletas/{id}	asistente	403	BOLA/IDOR
19	Listado completo de boletas	GET /boletas	asistente	403	RBAC
20	Cliente privado sin API Key	GET /seguridad/cliente	anónimo	401	API Key requerida
21	Cliente privado con API Key	GET /seguridad/cliente	cliente	200	—
22	Login sin API Key, cuerpo inválido	POST /auth/login	anónimo	400	Llega al validador
23	Mis boletas sin JWT ni API Key	GET /boletas/mias	anónimo	401	Token requerido
24	Mis boletas con JWT, sin API Key	GET /boletas/mias	asistente	200	Sin enviar clave
25	Esquemas en Swagger	GET /openapi.json	anónimo	200	Sin seguridad / Bearer / ApiKey
26	Preflight CORS	OPTIONS /auth/login	navegador	204	Origen localhost:5173
27	Catálogo sin credenciales	GET /eventos, /funciones…	anónimo	200	Seis rutas en 200
28	Ocupación sin datos ajenos	GET /funciones/1/ocupacion	anónimo	200	fila, localidadId, numero

#	Prueba	Endpoint	Rol	HTTP	Observación
29	Inactivos ocultos	GET /eventos	anónimo y admin	200	Público 10, mostrador 11
30	Crear evento sin JWT	POST /eventos	anónimo	401	Token requerido
31	Crear evento como asistente	POST /eventos	asistente	403	RBAC
32	Registro sin API Key, cuerpo inválido	POST /auth/registro	anónimo	400	Validación
33	Registro sin API Key, válido	POST /auth/registro	anónimo	201	Pidió admin, quedó asistente
34	config.js sin clave	GET /sala/config.js	navegador	200	Solo {"api":"/api"}
35	Restablecer sin API Key	POST /auth/restablecer-*	anónimo	401	Las dos rutas

Las verificaciones de la base (01 a 08) y las consultas del Paso 9 están en las secciones 2, 3 y 5. Todas dieron el resultado esperado.

8. La tanda completa y la salvaguarda

Cambiar qué credencial pide cada ruta afecta pruebas de bloques anteriores, así que corrimos todas las baterías del proyecto. Cinco pruebas viejas cambiaron, y en todas fue por el contrato nuevo, no para hacerlas pasar:

- las pruebas 22, 24 y 25 del Paso 8, que ahora esperan lo que dice la guía;
- «GET /api/eventos sin API Key responde 401» pasó a comprobar que la cartelera responde 200 sin credenciales;
- «Leer una boleta sin X-API-Key es 401» sigue en 401, pero ahora lo rechaza el JWT; solo cambió el texto.

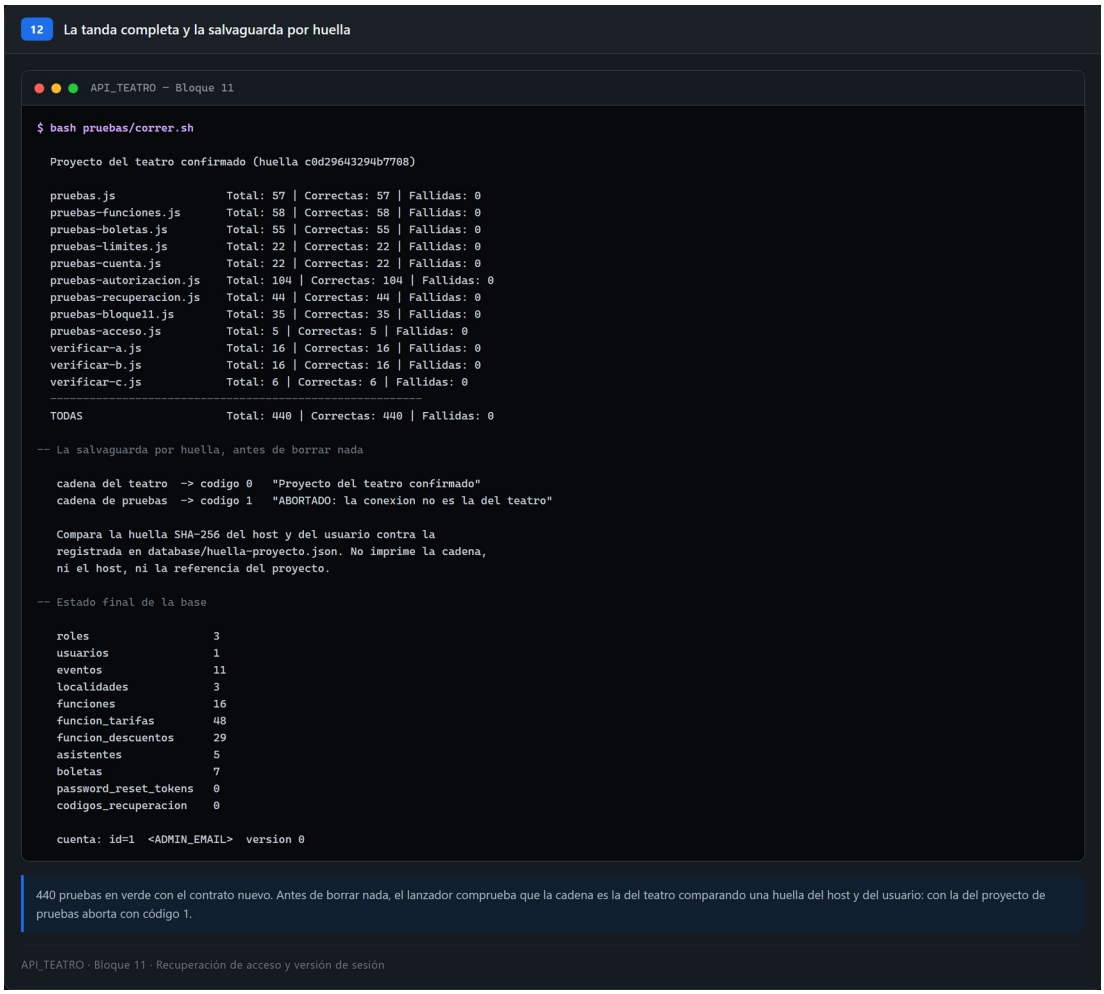


Figura 16. La tanda completa: 440 de 440, la salvaguarda por huella y el estado final de la base.

El total es **440 de 440**: las baterías de siempre, las 44 de recuperación, las 35 de este bloque y 5 de límites de acceso. Esas cinco van aparte porque necesitan el límite real de 10 intentos, y las demás hacen decenas de inicios de sesión legítimos.

Por qué la salvaguarda está dentro de preparar-base.js

Las baterías reinician la base para partir de datos limpios, y reiniciar significa borrar tablas. Una cadena de conexión no dice a qué proyecto pertenece: si se pega la equivocada en el `.env`, el script borra la base de otro proyecto sin dar ningún aviso. Eso ya nos pasó una vez.

Por eso, antes de abrir la conexión, el script calcula una huella SHA-256 del host y del usuario y la compara con la registrada en `database/huella-proyecto.json`. Ese archivo solo guarda la huella, nunca el host ni la contraseña.



Figura 17. Fragmento de preparar-base.js: la comprobación ocurre antes de cualquier sentencia.

Al principio la comprobación estaba solo en `correr.sh`, y correr `preparar-base.js --reiniciar` directamente se la saltaba. La movimos al propio script. Con la cadena del proyecto de pruebas aborta con código 1 y no ejecuta nada; con la del teatro continúa. Al terminar la tanda, la base queda con la semilla, un administrador y las dos tablas nuevas vacías.

Análisis estático y de dependencias

Semgrep no encontró nada: 0 hallazgos con 76 reglas sobre 85 archivos. `npm audit` reporta 4 vulnerabilidades moderadas en `glob` y `minimatch`, que llegan por `swagger-jsdoc` y `nodemon`. No las corregimos. La única solución que ofrece npm es bajar `swagger-jsdoc` a una versión anterior, y ninguna de las dos se puede alcanzar desde una petición: solo resuelven el patrón fijo `./src/routes/*.js` al arrancar, y `nodemon` es solo de desarrollo.

9. Lo que queda pendiente y observaciones

- **Las dos rutas de restablecimiento siguen con API Key**, igual que en la guía. Hoy nadie las llama desde el navegador. Cuando exista la interfaz web habrá que decidir si se abren como el login (apoyadas en el límite de intentos y en que el token y el código son de un solo uso) o si esos dos flujos se hacen por fuera del navegador.
- **El registro pide 10 caracteres sin complejidad**, mientras que las contraseñas nuevas de la recuperación piden 12 con mayúscula, minúscula y número. Es igual en el insumo de la guía, y lo dejamos así para no cambiar el contrato de un endpoint ya probado, pero recuperar una cuenta termina siendo más exigente que abrirla.

- **El correo del administrador es fácil de adivinar** y aparece en informes anteriores del repositorio. Con el login público es la mitad conocida de unas credenciales. Lo que lo protege es la contraseña larga y el límite de intentos; cambiarlo por una dirección no publicada queda como mejora.
- **CORS** acepta varios orígenes separados por comas. Para el desarrollo de la interfaz web, `ALLOWED_ORIGIN` queda con `http://localhost:3000,http://localhost:5173`.

El código está en el repositorio del proyecto, https://github.com/JuanK2550/API-TEATRO, rama `main`. Los cambios de este bloque quedaron en seis commits, separados por motivo para poder revertir uno sin arrastrar los otros:

- `2e2024a`: las tres migraciones y `preparar-base.js`;
- `cca84f7`: versión de sesión, servicios de recuperación, controlador y validadores;
- `24e5cec`: el reparto de credenciales, el filtrado público y Swagger;
- `89e365e`: las baterías nuevas, la salvaguarda y los ajustes a pruebas viejas;
- `3d088b5`: la salvaguarda dentro de `preparar-base.js` y el retiro de la clave web;
- `53b5373`: el informe del repositorio, el README, `.env.example` y los análisis.

Ninguno incluye el `.env`, y revisamos el diff completo buscando hosts, cadenas de conexión y tokens: no apareció ninguno.

10. Conclusión: RLS en PostgreSQL frente a los controles de Express

Este bloque nos dejó clara una diferencia que la guía repite varias veces: tener RLS habilitado en una tabla no protege nada por sí solo.

RLS decide qué filas puede tocar una conexión, y solo funciona si la conexión no se lo salta. La verificación 08 lo demuestra en el teatro: la API entra como `api_teatro`, sin `BYPASSRLS`, y por eso las tablas nuevas necesitaron su `GRANT` y su política. Sin ellos, la API no habría podido usarlas.

Express decide qué puede hacer cada persona, y eso RLS no lo sabe. Nuestra política es `USING (true)` para `api_teatro`: la API puede leer cualquier fila, y necesita poder hacerlo para responder «403, esa boleta no es tuya». Quién es cada quien lo deciden el JWT, `autorizarRoles` y las guardas de propiedad, como se vio en las pruebas 11, 18, 19 y 31.

Y hay una tercera pregunta que no responde ninguna de las dos: **qué es público**. Eso lo decide el dominio. La cartelera no tiene dueño y una boleta sí. Por eso el catálogo se abrió

sin credenciales, lo demás quedó con JWT y el navegador ya no necesita una clave que nunca fue secreta.

> **RLS** protege la base de una conexión equivocada.
> **Express** protege los datos de una persona equivocada.
> Un proyecto necesita las dos capas, y tener RLS habilitado no demuestra ninguna de las dos.